

DATA 266 Homework 1

DATA 266 — Homework 1

Personal Parameters (Section 0.1)

Parameter	Value
SID4	1605
SEED	1605
SLICE	605 (not used by HW1)
HP_ID	3 — Learning-rate-low
CLS_A	5 (not used by HW1)
CLS_B	3 (not used by HW1)

Part 1 — What are autoregressive models? (5 points)

An **autoregressive (AR) model** generates a sequence one element at a time, where each new element is predicted conditioned on the elements produced (or observed) so far. For a sequence $x_1, \dots, x_T$, it factorizes the joint probability by the chain rule, without an independence assumption between steps:

$$P(x_1, x_2, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_1, \dots, x_{t-1})$$

Each conditional is learned by a model (classical linear AR/ARIMA, an RNN, or a Transformer decoder), and generation proceeds left to right: predict/sample x_t , append it to the context, predict x_{t+1} , repeat. The model regresses on its own past outputs — hence “autoregressive.”

Real-world examples:

- **Classical time-series forecasting (AR/ARIMA):** predicting tomorrow’s stock price or temperature from the last k observed values.
 - **GPT-style large language models:** every output token is predicted conditioned on all previous tokens, one token at a time, left to right (e.g., ChatGPT, Claude, GPT-4).
 - **WaveNet / audio generation:** raw audio waveforms generated one sample at a time, each conditioned on previously generated samples.
 - **PixelRNN / PixelCNN:** images generated pixel by pixel, each pixel’s distribution conditioned on previously generated pixels in raster-scan order.
 - **Code completion (e.g., GitHub Copilot):** the next token is predicted from all preceding tokens in the file, the same autoregressive mechanism as chat LLMs.
-

Part 2 — Diabetes Dataset: Neural Network Implementation (10 points)

Dataset: Pima Indians Diabetes (768 rows, 8 numeric features, binary `Outcome`), loaded from a public mirror of the standard no-header UCI-derived CSV. Full pipeline, code, and executed outputs are in `neural_networks.ipynb`.

Preprocessing: physiologically-impossible zeros in `Glucose`, `BloodPressure`, `SkinThickness`, `Insulin`, `BMI` were recoded as missing and imputed with the training-set median; all features were standardized with a scaler fit only on the training split.

Split: 70% / 15% / 15% train/val/test, stratified on `Outcome`, `random_state = SEED = 1605`, reused identically for every model comparison below.

Configurations:

Model	Hidden layers	Learning rate	Epochs
Baseline (both frameworks)	[64, 32]	0.001	30
Modified — HP_ID = 3 (both frameworks)	[64, 32]	0.0003	30

Results — multi-seed test accuracy

Data split fixed at `SEED = 1605`; training seeds 1605, 1606, 1607.

Framework	Model	Test acc (mean $\pm$ std)
PyTorch	baseline	0.7529 $\pm$ 0.0050
PyTorch	modified (HP_ID=3)	0.7270 $\pm$ 0.0132
TensorFlow	baseline	0.7414 $\pm$ 0.0172
TensorFlow	modified (HP_ID=3)	0.7213 $\pm$ 0.0199

Loss curves (`SEED = 1605` run)

See `figures/pytorch_loss_curves.png` and `figures/tensorflow_loss_curves.png`. At epoch 30:

Framework	Model	Train loss	Val loss
PyTorch	baseline	0.382	0.419
PyTorch	modified	0.428	0.404
TensorFlow	baseline	0.363	0.428
TensorFlow	modified	0.421	0.408

Conclusion

The baseline consistently outperforms the HP_ID=3 modified model in **both** frameworks and at **all three** training seeds, by roughly 2-3 points of test accuracy, with the modified model also showing 2-4x higher seed-to-seed variance. The loss curves explain why: at epoch 30 the baseline's train loss is *below* its validation loss (0.382 vs 0.419 for PyTorch) — it has largely converged and is beginning to fit training-set-specific noise — while the modified model's train loss is still *above* its validation loss (0.428 vs 0.404) — it has not yet converged. TensorFlow shows the identical qualitative pattern (0.363/0.428 baseline vs 0.421/0.408 modified), which is expected since only the learning rate changed between baseline and modified; architecture, epoch budget, batch size, and data split are all identical.

This is **underfitting relative to baseline caused by an insufficient effective training budget**, not a regularization benefit: a learning rate lowered 3.3x (0.001 $\rightarrow$ 0.0003) takes smaller gradient steps, so within the same fixed 30-epoch budget it simply hasn't traveled as far toward a minimum. The modified model's

higher cross-seed variance is consistent with this — it has moved less far from its (seed-dependent) random initialization, so initialization noise still shows up in the final result. Given a longer epoch budget, the lower learning rate would likely close or reverse this gap; within the 30-epoch budget specified by `HP_ID=3`, it is strictly worse than baseline in this experiment. Per the assignment, this is still the required modified model regardless of whether it outperforms baseline.

Part 3 — CUDA Matrix Multiplication (5 points)

Status: pending Colab GPU run. This report was assembled on a machine with no NVIDIA GPU (`nvcc` not installed). `matmul.cu` and `cuda.ipynb` are complete and ready to run — `cuda.ipynb` should be executed top-to-bottom on Google Colab with a GPU runtime, and the results below replaced with the real output before submission.

Kernel design (blocks and threads): the output matrix is tiled into 16x16-thread blocks (256 threads/block); the grid is a 2D array of blocks sized to cover the whole $N \times N$ output, so each thread computes exactly one output element `C[row][col]`. Threads in a block cooperatively stage 16x16 sub-tiles of the two input matrices into `__shared__` memory (synchronized with `__syncthreads()`), turning most global-memory reads into fast shared-memory reads before accumulating the dot product — the standard tiled shared-memory `matmul` pattern. Full detail is commented in `matmul.cu`.

Profiler: Nsight Systems (`nsys profile --stats=true`) — `nvprof` was removed from the CUDA toolkit starting with CUDA 11 and is not present on current Colab images, so `nsys` (the tool that replaced it) is used instead, as `cuda.ipynb` documents.

Required table (to be filled in from the Colab run)

Matrix size	CPU (ms)	GPU kernel (ms)	H2D+D2H (ms)	Speedup
256	<i>pending</i>	<i>pending</i>	<i>pending</i>	<i>pending</i>
1024	<i>pending</i>	<i>pending</i>	<i>pending</i>	<i>pending</i>
4096	<i>pending</i>	<i>pending</i>	<i>pending</i>	<i>pending</i>

Crossover analysis (to be finalized with real numbers)

The smallest matrix size at which GPU end-to-end time (kernel + H2D + D2H) is expected to beat the CPU baseline is somewhere between 256 and 1024 in this design, though the exact crossover must be read off the real Colab measurements above. The crossover is not at size 0 because launching a kernel and moving data across PCIe both carry fixed, largely N -independent overhead (CUDA launch/context latency, transfer setup latency), while transfer time scales with N^2 (bytes moved) and only the useful compute scales with N^3 (`matmul` FLOPs). At small N there isn't enough $O(N^3)$ parallel work to amortize that fixed overhead and the $O(N^2)$ transfer cost, so the CPU — which pays none of that overhead — can win; only once N is large enough for the $O(N^3)$ term to dominate does the GPU's much larger core count start to pay for the round trip across PCIe.

Repository note

Standing-requirement artifacts (`RUN_LOG.txt`, `METRICS.md`, `AI_USE.md`, `checkpoints/`, `figures/`) accompany this report in the same repository, per Section 0.3.